College Project Report

Web Log Analyzer

A Browser-Based Tool for Reviewing JSON and JSONL Logs

Submitted by

Akshay S R
D Nithin
D Kiran
Rahul
Abhilash V
G Vinay

Project Report

Abstract

Application log files contain records of events and problems produced while software runs. When a file becomes large, finding errors, repeated messages, and relevant records by reading each line manually can be slow. The Web Log Analyzer is a browser-based application designed to make this review process easier. It loads JSON objects, JSON arrays, and JSONL files from a user's computer; summarizes severity and event information; supports search and filtering; allows individual records to be expanded; and provides a downloadable JSON report. The application includes sample files for checking different input formats and repeated-error cases. Its analysis is performed in the browser on the user's device. This report describes the project's motivation, objectives, workflow, implementation approach, sample files, and possible future improvements.

Contents

Abstract **i**

1 Introduction **1**

1.1 Project Scope 1

2 Problem Statement and Motivation **2**

3 Project Objectives **3**

4 System Overview **4**

4.1 Technologies 4

4.2 Workflow 4

5 Features and Processing Logic **6**

5.1 Supported Input 6

5.2 Summary and Record Review 6

5.3 High-Level Algorithm 6

5.4 Data Handling 7

6 Sample Files and Practical Testing **8**

7 Advantages and Limitations **9**

7.1 Advantages 9

7.2 Current Scope and Limitations 9

8 Conclusion and Future Work **10**

Reference **11**

Chapter 1

Introduction

Software applications produce log records to describe events such as service startup, completed requests, warnings, and errors. These records help developers understand application behavior and investigate problems. However, logs often combine normal activity with important events, making manual review difficult.

The Web Log Analyzer provides a simple browser interface for loading and exploring log files. A user can choose a built-in sample or provide a JSON or JSONL file, review summary information, search and filter the records, expand a record to inspect its fields, and download the current analysis as a JSON report.

1.1 Project Scope

This report covers the browser-based Web Log Analyzer shown in the supplied presentation. The application focuses on local log-file review. It does not require a remote logging service or a separate database for its described workflow.

Chapter 2

Problem Statement and Motivation

Large log files may contain many records, while the events that matter to an investigation can be difficult to locate. Errors and repeated events may be mixed with routine messages. Searching line by line requires the reader to inspect each record individually.

A practical log-review interface should make it possible to:

- load common structured log formats from a local computer;
- find records using text search and filters;
- recognize severity, event, and repeated-error patterns;
- inspect the complete fields of an individual record; and
- save the results for later review.

Chapter 3

Project Objectives

The project has five main objectives:

1. Make log analysis accessible through a web browser.
2. Support JSON and JSONL log files selected from the user's computer.
3. Provide search and severity or event filtering.
4. Let users expand a record to inspect all of its fields.
5. Allow the current analysis to be downloaded as a JSON report.

Chapter 4

System Overview

4.1 Technologies

The application uses a small set of widely used technologies:

Technology	Role in the application
HTML	Defines the page structure and controls.
CSS	Defines the layout, colors, and visual presentation.
JavaScript	Reads files, parses records, calculates summaries, applies filters, and updates the page.
JSON and JSONL	Provide the supported structured log-file formats.
Python HTTP server	Serves the local webpage during development and use.
Visual Studio Code	Provides an environment for opening and editing the project files.

4.2 Workflow

The normal workflow is:

1. Open the analyzer page in a browser.
2. Load a built-in sample or select or drop a local JSON or JSONL file.
3. Parse the input into individual records.
4. Review summaries and use search or filters to narrow the record list.
5. Expand records when additional fields are needed.
6. Download the current analysis as a JSON report.

The included Python HTTP server is used to serve the application locally. The log-file review itself is performed in the browser on the user's device.

Chapter 5

Features and Processing Logic

5.1 Supported Input

The application accepts three common JSON shapes:

- a single JSON object representing one record;
- a JSON array containing multiple records; and
- JSONL, where each non-empty line contains one JSON record.

If a JSONL line is malformed, the application can report that line as an issue while retaining valid records for analysis. This helps a user continue reviewing a file that contains a damaged entry.

5.2 Summary and Record Review

The analyzer presents summary information about the loaded records, including severity and event counts. It also groups matching error messages to make recurring problems easier to notice. The record explorer supports text search and severity or event filters. Expanding a row reveals the full fields stored in that record. A JSON report can be downloaded after review.

5.3 High-Level Algorithm

The following pseudocode describes the main processing flow:

Listing 5.1: High-level log analysis flow

```
read selected file as text

```

```
try:
    parse complete text as JSON
    if it is an object:
        add it as one record
    if it is an array:
        keep object entries and report non-object entries
otherwise:
    for each non-empty line:
        try to parse the line as JSON
        keep valid object records
        record malformed or non-object lines as issues

count records by severity and event
group error records by normalized message
apply the selected search and filters
display summaries and matching records
export the current summary as JSON when requested
```

5.4 Data Handling

The input file is read by the browser application for analysis on the user's device. The described workflow does not upload the log file to a remote analysis service. Users should still follow their organization's rules when handling sensitive logs.

Chapter 6

Sample Files and Practical Testing

The project includes four sample files. Each is intended to demonstrate a different input or review case.

Sample file	Contents	Review purpose
`health_check.json`	One JSON object.	Basic file loading.
`batch_jobs.json`	An array of records.	Reviewing multiple records.
`api_gateway.jsonl`	JSONL with a malformed line and repeated errors.	Error investigation and malformed-line handling.
`error_burst.jsonl`	Repeated database errors.	Checking repeated-error grouping.

These sample files provide practical checks for the supported formats, search and filtering workflow, expanded record details, malformed-entry reporting, and repeated-error review. The project presentation identifies these as sample scenarios; this report does not claim benchmark performance or production testing.

Chapter 7

Advantages and Limitations

7.1 Advantages

- The browser interface makes basic log review accessible without a complex setup.
- Support for both JSON and JSONL covers multiple common log layouts.
- Search and filters help narrow a large record list.
- Expanded rows expose the full data for a selected record.
- Local browser processing keeps the described file-review workflow on the user's device.

7.2 Current Scope and Limitations

The application is intended for structured JSON and JSONL input. It is not presented as a general parser for plain-text logs or every logging format. The supplied presentation does not provide measured performance results for very large files. Error-count charts, stronger grouping, additional file formats, CSV export, and dashboard summaries are proposed as future enhancements rather than current features.

Chapter 8

Conclusion and Future Work

The Web Log Analyzer addresses the difficulty of reviewing structured application logs manually. It combines local file loading, summaries, search, filters, record inspection, and JSON report download in a browser-based workflow. The sample files give users a way to explore both ordinary records and error cases.

Future development could add error-count charts, improve repeated-message grouping, support additional log formats, export CSV files, and provide broader dashboard summaries. Testing with larger real-world files would also help establish performance limits and guide future improvements.

Reference

1. Project team, *Web Log Analyzer*, 10-slide project presentation supplied as the source for this report, 2026.